

Mini-Project-1 Tutorial

本教程展示了如何基于学校卓越中心的 NPU 算力平台，为 llama.cpp 上的 CANN 后端新增算子

创建环境

有两种方式可以获得一个交互式运行环境，因为平台分为 SCOW AI 和 SCOW HPC。其中 SCOW AI 是基于容器的算力集群，SCOW HPC 是基于裸金属服务器的算力集群。

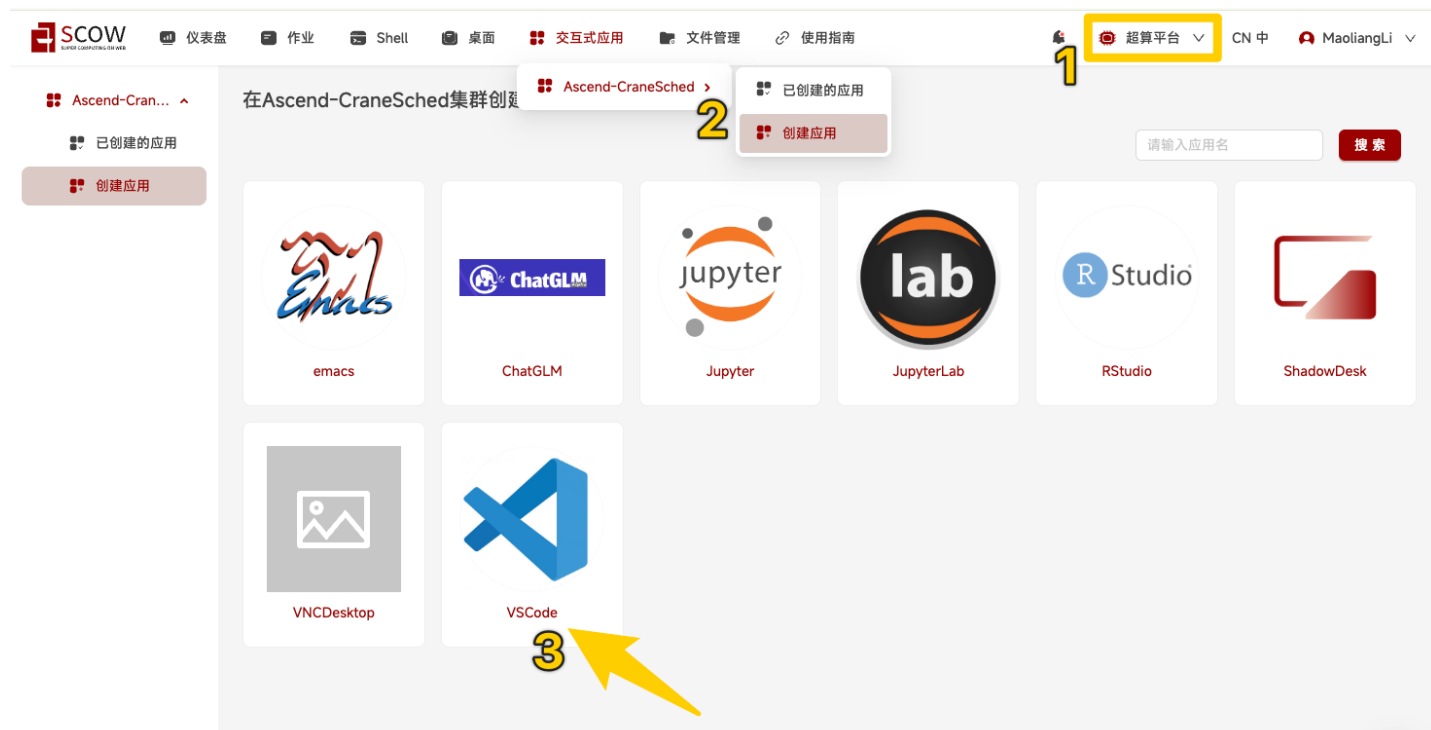
1. 超算平台（后续主要使用）

登录后，选择超算平台 → 交互式应用 → 创建 VSCode 应用 → 已创建的应用。

创建 VSCode 应用时，无需修改默认配置，等待一会后点击 **进入** 即可打开网页版的 VSCode。

基于网页版 VSCode 的内置终端，可以运行命令进行环境配置和代码获取。

该方案的好处是，不需要关心数据保存的问题。应用实例会自动在你的 home 目录下启动，时长到了之后应用进程会关闭，但下次启动依然会使用你的 home 目录。



SCOW

仪表盘

作业

Shell

桌面

交互式应用

文件管理

使用指南

超算平台

CN 中

MaoliangLi

Ascend-Cran...

已创建的应用

创建应用

创建VSCode

* 作业名:

pkuhpc-VSCode-20251014-105602

* 账户:

1106183127

C

* 分区:

cntrain

C

* QOS:

normal

▼

* 节点数:

1

* 单节点加速卡卡数:

1

* 最长运行时间:

30

分钟

▼

* VSCode版本:

4.95.3

▼

其他sbatch参数:

比如: --gpus gres:2 --time 10

总加速卡卡数: 1

总CPU核心数: 22

总内存容量: 246 GB

取消

提交

<

SCOW

仪表盘

作业

Shell

桌面

交互式应用

文件管理

使用指南

超算平台

CN 中

MaoliangLi

Ascend-Cran...

已创建的应用

创建应用

集群Ascend-CraneSched交互式应用

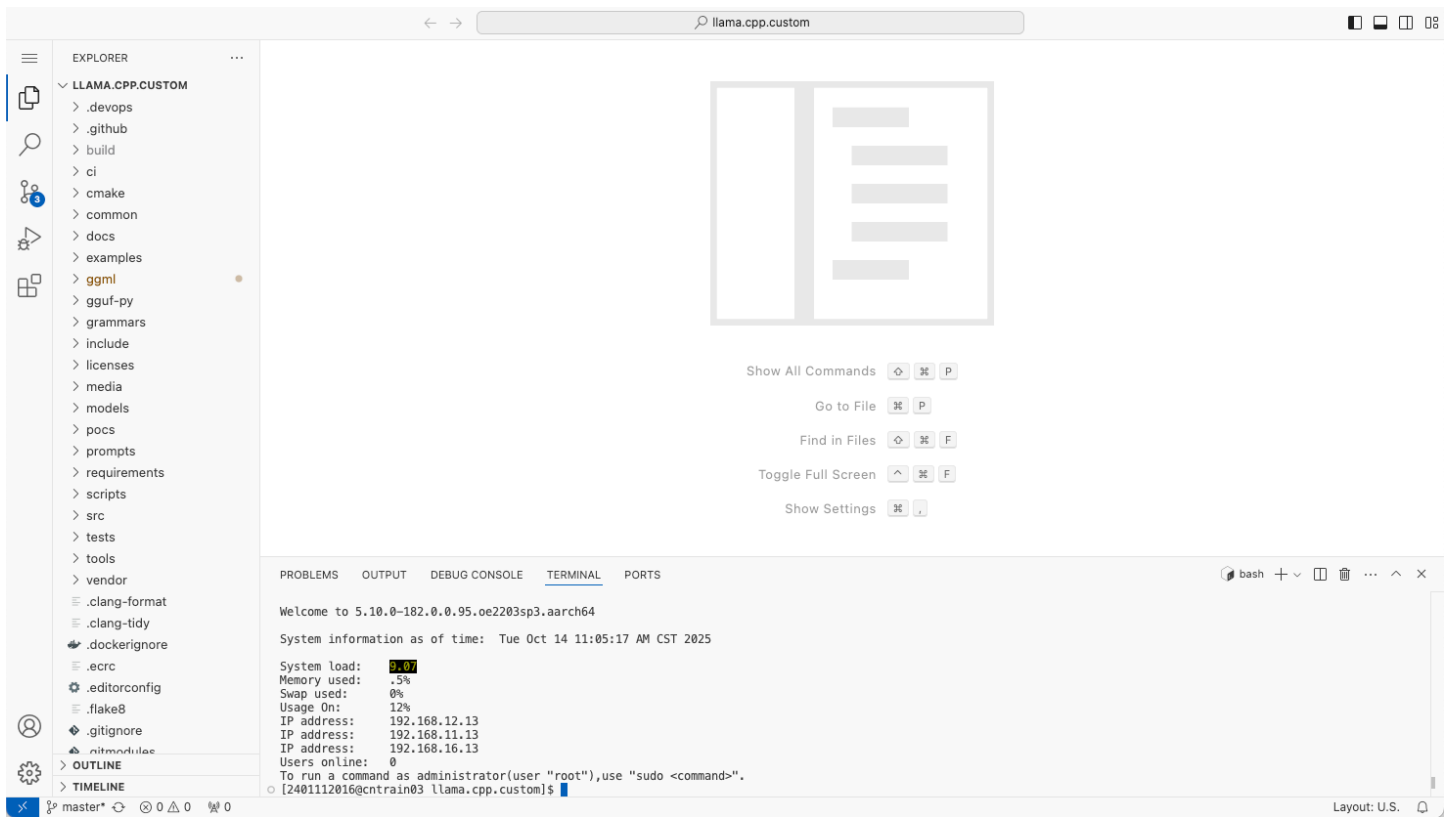
作业名:

搜索

刷新

☐ 只展示未结束的作业

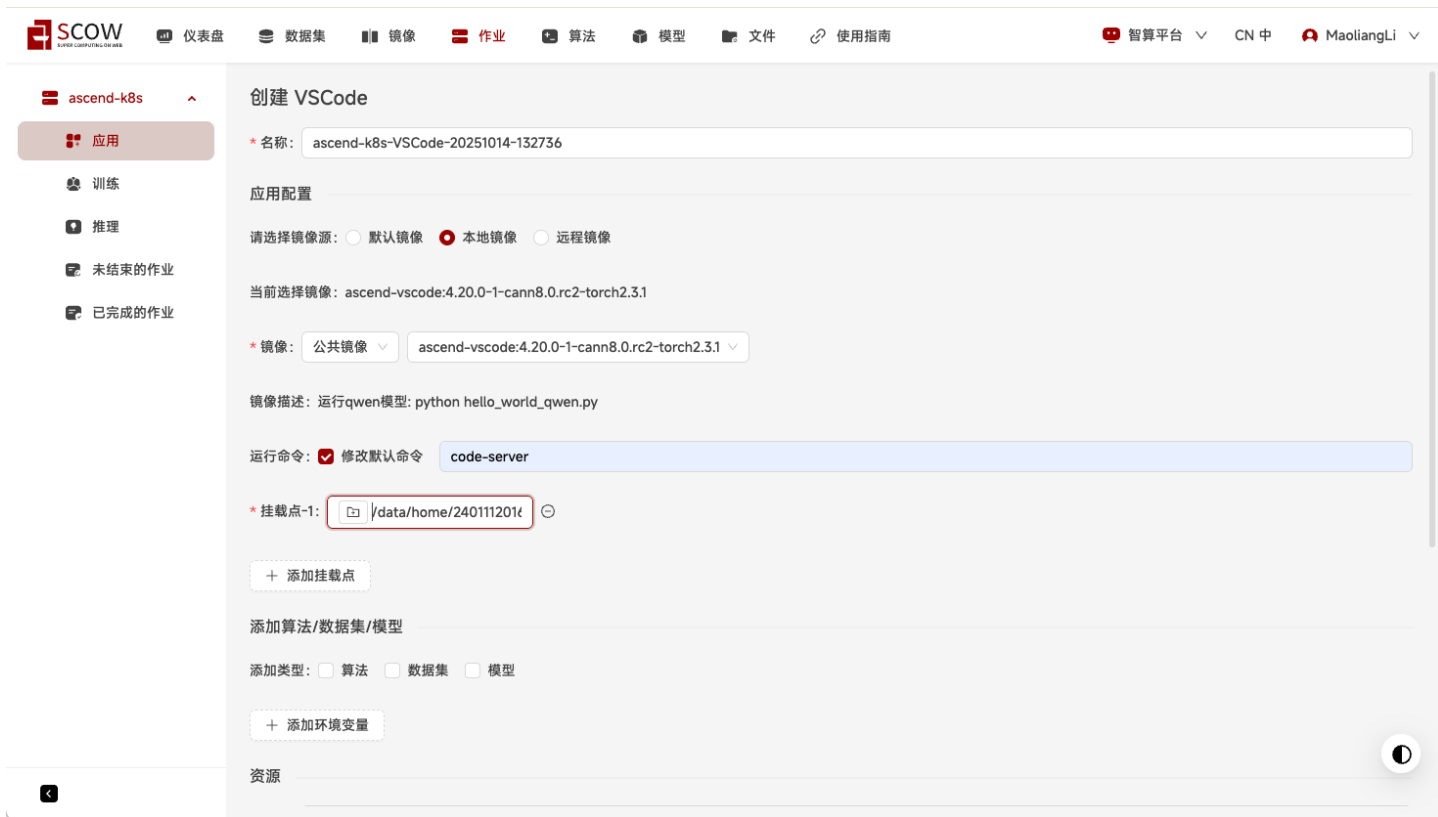
作业ID	作业名	应用	提交时间	状态	剩余时间	操作
11949	pkuhpc-VSCode-20251014-110227	VSCode	2025-10-14 11:02:38	RUNNING	29:47	进入 📁 📄 ⏻
11940	pkuhpc-VSCode-20251013-171348	VSCode	2025-10-13 17:14:06	ENDED		📁
11939	pkuhpc-VSCode-20251013-151220	VSCode	2025-10-13 15:12:30	ENDED		📁
11938	pkuhpc-VSCode-20251013-143953	VSCode	2025-10-13 14:40:16	ENDED		📁



2. 智算平台（助教没用这个）

在创建 VSCode 镜像的时候，默认镜像没有预装 CANN，因此建议选用本地镜像-公共镜像-vscode+cann 的镜像。想要保存数据，记得挂载自己的 home 目录并在其中进行操作。界面有个小 bug，就是它没有默认命令，一般写 `code-server` 就行。





另外，需要提醒的是，镜像的环境（Debian12）比较基础，编译器工具链、cmake 等都需要自行安装。由于助教懒得再尝试新的环境，这部分的配置请自行探索。

其他

如有问题，请先参考SCOW平台的使用指南（<https://docs.qq.com/doc/DQnFXRIZvWUZRY0dU?token=6f9910bc-4f59-4962-9269-431116e9cad8>）。如果无法解决，请联系老师和助教协助。

编译部署 llama.cpp

首先获取基础代码。

需要注意的是，请勿直接使用 llama.cpp 的官方仓库代码，虽然官方支持 CANN 后端，但是实测由于版本兼容问题，无法运行，会报 Context Pointer Null 错误。

Code block

```
1 git clone https://github.com/noemotiovon/llama.cpp.git
2 cd llama.cpp
```

然后编译运行。编译会比较缓慢，大约需要5分钟。

Code block

```
1 # 最好先加载一下 CANN 的环境
2 source /usr/local/Ascend/ascend-toolkit/set_env.sh
3 source /usr/local/Ascend/nnaI/atb/set_env.sh
4 # cmake 编译
```

```
5  cmake -B build -DGGML_CANN=on -DCMAKE_BUILD_TYPE=release
6  cmake --build build --config release -j
```

完成编译后，执行 test 来检查是否可以正常运行。

当然，如果你愿意的话，也可以下载一个 gguf 格式的大模型来跑一跑

Code block

```
1  ./build/bin/test-backend-ops test -b CANN0 -o ADD
```

修改算子流程

具体任务中，如何修改算子可以参考 [issue](#) 下面的说明。

这里给出一个新增 LayerNorm 算子的 Step-by-Step 流程。

0. 代码树结构

主要的代码分布在两个目录下面，`/src` 实现了 llama cpp 的推理框架部分，在本次 project 中基本不需要修改；`/ggml` 下面放的是算子实现，本次任务主要涉及 `/ggml/src/ggml-cann/`（需要修改）和 `/ggml/src/ggml-cpu`（参考功能实现）。

在 `ggml-cann` 中，`ggml-cann.cpp` 实现公共的包装代码，`acl_tensor` 实现了 ggml 和 cann 的数据类型转换工具代码，`aclnn_ops` 实现了所有的算子。

1. 修改算子注册

打开 `/ggml/src/ggml-cpu/ggml-cann.cpp`，在 `ggml_backend_cann_supports_op` 中，写了一个长长的 `switch(op->op)`，llama.cpp 有一个预定义的算子集枚举，如果这个 switch 返回 true 则说明该算子被该后端所支持。

你需要检查 `case GGML_OP_NORM:` 是否返回 true，如果不是，修改一下。当然，所有正经的后端实现必然支持 LayerNorm 操作，因此，你会在2470行左右发现 `case GGML_OP_NORM:` 导向 true。

2. 增加算子的 forward 调用

打开 `/ggml/src/ggml-cpu/ggml-cann.cpp`，在 `ggml_cann_compute_forward` 中，写了一个长长的 `switch(dst->op)`，llama.cpp 有一个预定义的算子集枚举，这里会调用对应的算子实现。

仿照其他已经写好的算子，你可以在一长串 case 里面增加一段（当然，这已经在1170行左右写好了，如果是你自己需要新增算子支持，需要写一个类似的）：

Code block

```

1  case GGML_OP_NORM:
2      ggml_cann_norm(ctx, dst);
3      break;

```

其中，`ggml_cann_norm` 是你用 CANN 实现的 LayerNorm 算子，在下一节中将说明具体如何做。

3. 实现算子

在 `/ggml/src/ggml-cpu/aclnn_ops.h` 中，增加函数声明，下面的代码已经实现在 L188。

Code block

```

1  void ggml_cann_norm(ggml_backend_cann_context& ctx, ggml_tensor* dst);

```

在 `/ggml/src/ggml-cpu/aclnn_ops.cpp` 中，新增函数实现。

该函数直接调用 CANN 实现的 `aclnnLayerNorm` 算子来实现操作，因此主要的工作是转换数据格式以及设置参数。

如果你不知道一个算子应该完成什么功能，你可以去别的后端里面翻一翻已经实现好的算子，例如 cpu 后端的算子实现是比较全面的，你可以找到代码切出来问一问 GPT。

Code block

```

1  void ggml_cann_norm(ggml_backend_cann_context& ctx, ggml_tensor* dst) {
2      /*
3       ctx 是 CANN 上下文，保存 stream、handle 等资源。
4       dst 是 GGML 的输出张量，同时也携带了输入张量指针和算子参数。
5       */
6      ggml_tensor* src = dst->src[0];
7      /*
8       GGML 的算子张量链里，dst->src[0] 指向该 LayerNorm 的输入张量。
9       */
10     aclTensor* acl_src = ggml_cann_create_tensor(src);
11     aclTensor* acl_dst = ggml_cann_create_tensor(dst);
12     /*
13     把 GGML 的 src / dst 转成 CANN 的 aclTensor 描述符 (ggml_cann_create_tensor 定义
14     在 acl_tensor.cpp 中)。
15     内部会把 GGML 的 ne (元素数)、nb (stride)、data 通过 aclCreateTensor 映射到 CANN
16     的格式。
17     */
18     float eps;
19     memcpy(&eps, dst->op_params, sizeof(float));
20     /*
21     LayerNorm 需要一个 eps 防止除零。
22     GGML 把 eps 存在 dst->op_params 前 4 字节里，这里直接按位拷出来。

```

```

21  */
22      std::vector<int64_t> normData = {dst->ne[0]};
23      aclIntArray* norm = aclCreateIntArray(normData.data(), normData.size());
24  /*
25  CANN 的 aclnnLayerNorm 要求把“归一化维度”以 aclIntArray 形式传进去。
26  对最后一维做归一化，所以直接取 dst->ne[0] (GGML 把最后一维放在第 0 维)。
27  */
28      GGML_CANN_CALL_ACLNN_OP(ctx, LayerNorm, acl_src, norm, nullptr, nullptr,
29                              eps, acl_dst, nullptr, nullptr);
30      ggml_cann_release_resources(ctx, norm, acl_src, acl_dst);
31  /*
32  释放刚才创建的 aclIntArray 和 aclTensor 描述符，防止泄漏。
33  内部依次 aclDestroyIntArray(norm)、aclDestroyTensor(acl_src)、
34  aclDestroyTensor(acl_dst)。
35  */
36  }

```

该 API 的 AscendCL 文档为 [aclnnLayerNorm&aclnnLayerNormWithImplMode](#)，可以看见，`GGML_CANN_CALL_ACLNN_OP` 宏中填入的参数和它的一阶段调用是完全一致的。

- 第 3、4 个 `nullptr` 分别是 `scale` 和 `bias`，这里用不到，CANN 默认为 1/0。
- 最后两个 `nullptr` 是可选的输出数据 `meanOutOptional` 与 `rstdOutOptional`，这里用不到。

4. 重新编译并测试

Code block

```

1  cmake --build build --config release
2  ./bin/test-backend-ops test -b CANN0 -o NORM

```

如果没有报错，说明算子实现正确。